

Modulo Extra

Git y buenas practicas en desarrollo agentic

Branches · Commits · Worktrees · PRs · CI · CODEOWNERS · Seguridad
Desarrollo de Software IX · Código 1493 · Actualizado a mayo de 2026

Proposito del modulo

Explicar como usar Git con rigor cuando humanos y agentes de IA modifican el mismo repositorio: ramas pequenas, commits auditables, worktrees aislados, PRs con revision humana, CI obligatorio y reglas que evitan cambios destructivos.

Tabla de contenido

1. Por que Git cambia en un ambiente agentic	3
2. Modelo mental: area de trabajo, index e historial	3
3. Ramas pequenas y nombres utiles	4
4. Commits auditables	4
5. Worktrees: aislamiento para agentes paralelos	5
6. Pull requests como frontera de calidad	6
7. Branch protection, CODEOWNERS y CI	7
8. Conflictos, rebase y merge	8
9. Reglas para AGENTS.md en Git	8
10. Caso guia: cancelar reserva en MERN	9
11. Checklist production ready	9
12. Fuentes oficiales consultadas	9
13. Cierre	10

1. Por que Git cambia en un ambiente agentic

Git ya era el registro de decisiones del equipo. En desarrollo agentic se vuelve aun mas importante porque el repositorio puede recibir cambios de humanos, agentes locales, agentes remotos, automatizaciones, bots de dependencias y CI. Si Git se usa mal, el agente acelera el desorden: commits gigantes, ramas ambiguas, cambios sin revision, conflictos repetidos y secretos expuestos.

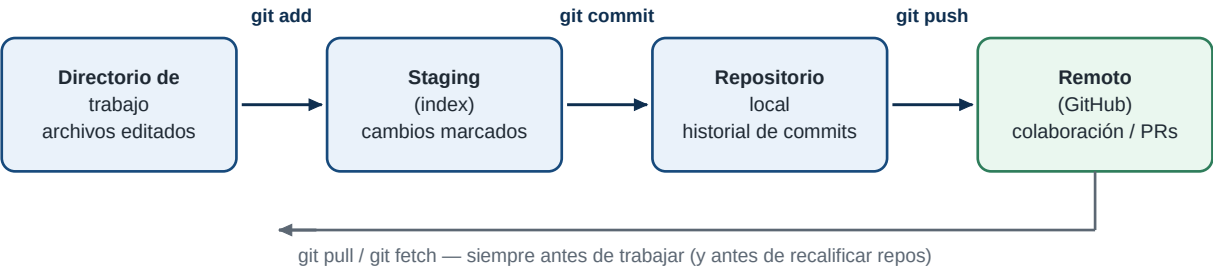
Idea central

El agente puede escribir codigo, pero Git debe conservar la trazabilidad. Cada cambio debe responder: que se hizo, por que se hizo, quien lo reviso, que pruebas pasaron y como revertirlo.

Practica	Uso tradicional	Uso agentic
Ramas	Aislar features.	Aislar tareas humanas, sesiones de agente y experimentos.
Commits	Guardar puntos de avance.	Crear evidencia revisable, con cambios pequenos y coherentes.
Worktrees	Trabajar varias ramas a la vez.	Ejecutar varios agentes sin pisar archivos ni servidores.
Pull requests	Revisar antes de mezclar.	Frontera obligatoria entre codigo generado y main.
CI	Detectar regresiones.	Validar que el agente no produjo solo una solucion superficial.

2. Modelo mental: area de trabajo, index e historial

Git separa tres momentos: archivos modificados en el working tree, seleccion de cambios en el index/staging area y commits en el historial. El agente debe trabajar dentro de ese modelo. Antes de editar debe mirar el estado; antes de commitear debe revisar diff; antes de subir debe ejecutar pruebas.



Comando	Pregunta que responde	Uso con agente
git status	Que cambio y que falta registrar.	Primera y ultima accion de una sesion.
git diff	Que cambio sin stage.	Revisar linea por linea antes de aceptar.
git diff --staged	Que entrara al commit.	Evitar mezclar cambios no relacionados.
git log --oneline --graph	Como llegamos aqui.	Orientar al agente antes de rebase o merge.

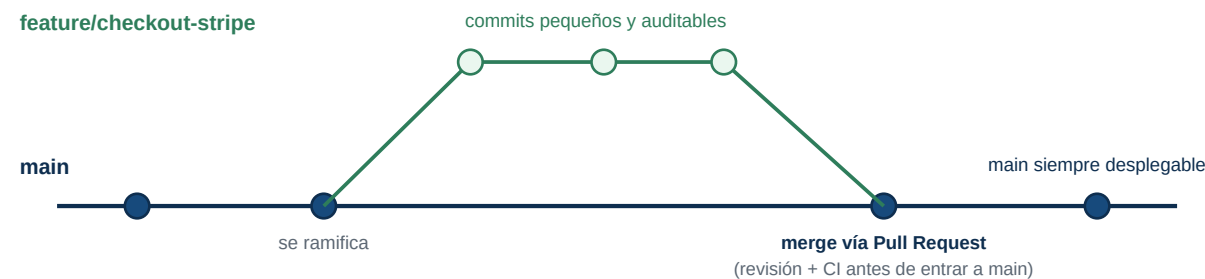
Comando	Pregunta que responde	Uso con agente
git restore	Como descartar un cambio puntual.	Usar con precision; no destruir trabajo ajeno.

Regla operativa

El agente nunca debe asumir que el working tree esta limpio. Si hay cambios existentes, debe identificarlos y trabajar con ellos sin revertirlos, salvo instruccion explicita.

3. Ramas pequenas y nombres utiles

Una rama representa una intencion. En trabajo agentico conviene que cada rama sea pequena y facil de revisar. Un agente no deberia acumular un redisenio, una migracion, cambios de estilo, pruebas y documentacion no relacionada en la misma rama.



Tipo de rama	Ejemplo	Cuando usarla
feature	feature/reservas-cancelacion	Nueva capacidad de usuario.
fix	fix/login-token-expirado	Correccion de bug reproducible.
chore	chore/actualizar-deps-test	Mantenimiento sin cambio funcional directo.
docs	docs/guia-api-reservas	Documentacion o material docente.
agent	agent/issue-42-validacion	Trabajo aislado de un agente o experimento.

Reglas para ramas en agentes

- Una rama por objetivo verificable.
- No trabajar en main directamente.
- Sincronizar con la rama base antes de iniciar trabajo largo.
- Eliminar ramas remotas cerradas si el flujo del equipo lo permite.
- Usar worktrees para trabajo paralelo en lugar de cambiar de rama sobre archivos sucios.

4. Commits auditables

Un commit debe contar una decision tecnica. En un entorno con agentes, los commits pequenos permiten detectar exactamente donde se introdujo un error. El historial no debe convertirse en un volcado de cambios

generados.

Buena practica	Ejemplo	Motivo
Mensaje imperativo	fix(auth): validate expired JWT	Explica la accion aplicada.
Scope claro	feat(reservas): add cancellation endpoint	Ubica el area afectada.
Un tema por commit	modelo + ruta + prueba del mismo caso	Facilita revertir y revisar.
No commitear ruido	sin logs, screenshots temporales o build outputs	Evita ensuciar el historial.
Firmar/verificar si aplica	commit signing en repos sensibles	Aumenta confianza de autoria.

Plantilla de commit para cambios con agente

```
tipo(scope): resumen breve

Contexto:
- Problema o ticket relacionado.

Cambio:
- Backend: ...
- Frontend: ...
- Tests: ...

Verificacion:
- npm test
- npm run build
- flujo manual revisado
```

5. Worktrees: aislamiento para agentes paralelos

Git worktree permite tener varios directorios de trabajo conectados al mismo repositorio. La documentacion oficial de Git lo describe como multiples working trees asociados a un repositorio. En mayo de 2026, Git 2.54 mantiene este mecanismo como una herramienta clave para trabajar varias ramas a la vez. Claude Code tambien documenta worktrees para correr sesiones paralelas sin que los cambios colisionen.

Escenario	Sin worktree	Con worktree
Dos agentes editan el repo	Se pisan archivos o ramas.	Cada agente usa su rama/directorio.
Bugfix urgente	Hay que stash o interrumpir feature.	Se abre worktree limpio para fix.
Review de PR	Se cambia de rama y se ensucia entorno.	Se revisa en directorio aislado.

Escenario	Sin worktree	Con worktree
Experimento	Riesgo de mezclar cambios.	Se descarta worktree si no sirve.

Comandos base

```
# Crear rama aislada para un agente

git fetch origin

git worktree add ../repo-agent-issue-42 -b agent/issue-42 origin/main


# Ver worktrees activos

git worktree list


# Eliminar cuando este limpio y cerrado

git worktree remove ../repo-agent-issue-42

git branch -d agent/issue-42
```

Cuidado practico

Los worktrees comparten el repositorio Git, pero no comparten necesariamente dependencias, variables locales ni servidores. En proyectos web, asigne puertos distintos por worktree para evitar choques entre agentes.

6. Pull requests como frontera de calidad

El pull request es donde el código agentico deja de ser una propuesta local y se convierte en candidato a integrarse. El PR debe incluir contexto, cambios, pruebas y riesgos. La revisión humana no es opcional cuando el agente toca autenticación, base de datos, pagos, despliegue, permisos o datos sensibles.

Seccion de PR	Debe incluir	Senal de alerta
Resumen	Problema y solución en 3-5 líneas.	Resumen generico sin relación al ticket.
Cambios	Archivos/capas principales.	Lista enorme sin criterio.
Pruebas	Comandos ejecutados y resultado.	No se ejecutaron pruebas.
Riesgos	Migraciones, auth, datos, UX, compatibilidad.	Dice 'sin riesgos' por defecto.
Capturas	UI antes/después si aplica.	Cambio visual sin evidencia.

Plantilla de PR para agente

```
## Contexto

Issue/tarea:


## Cambios principales
```

```
- Backend:

- Frontend:

- Datos/config:


## Verificacion

- [ ] lint

- [ ] tests

- [ ] build

- [ ] prueba manual o navegador


## Riesgos y rollback

- Riesgo:

- Como revertir:


## Uso de agente

Herramienta/agente usado:

Revision humana pendiente:
```

7. Branch protection, CODEOWNERS y CI

En GitHub, las ramas protegidas y los rulesets permiten exigir revisiones, estados de CI, restricciones de push y otras reglas antes de mezclar a main. CODEOWNERS asigna responsabilidad sobre carpetas o archivos y puede requerir aprobacion de quienes conocen esa parte. En repos con agentes, estas reglas son guardrails de produccion.

Control	Configuracion recomendada	Por que importa con agentes
Protected branch	No push directo a main; PR requerido.	Impide que el agente salte la revision.
Required checks	lint, test, build, e2e critico.	Detecta codigo que solo parece correcto.
CODEOWNERS	Auth, DB, infra y workflows con propietarios.	Cambios sensibles pasan por experto.
Dismiss stale approvals	Revisiones caducan si cambian commits.	Evita aprobar codigo que luego fue modificado.
Actions permissions	Token minimo y secretos protegidos.	Reduce impacto de PRs maliciosos o inseguros.

Regla de seguridad

Un agente puede proponer cambios en workflows CI/CD, pero no deberia aprobarlos ni desplegarlos sin revision humana. Cambios en .github/workflows, scripts de deploy o permisos requieren doble cuidado.

8. Conflictos, rebase y merge

Los conflictos aumentan cuando varios agentes trabajan en paralelo. La solución no es ocultarlos, sino reducir el tamaño de ramas y sincronizar frecuentemente. Rebase puede limpiar historial local; merge puede conservar historia compartida. La decisión depende del flujo del equipo.

Operacion	Uso	Precaucion
git fetch	Traer cambios sin mezclar.	Preferible antes de decidir merge/rebase.
git merge origin/main	Integrar base preservando merge commit.	Revisar conflictos con pruebas completas.
git rebase origin/main	Reaplicar commits sobre base actual.	No reescribir ramas compartidas sin acuerdo.
git cherry-pick	Traer commit puntual.	Evitar duplicar cambios si luego se mergea rama original.
git revert	Deshacer en historial publico.	Mejor que reset en ramas compartidas.

Protocolo de conflicto con agente

- 1 Detener cambios nuevos hasta entender el conflicto.
- 2 Leer ambos lados y la intención del PR/base.
- 3 Resolver mínimo necesario; no reescribir lógica no relacionada.
- 4 Ejecutar pruebas afectadas.
- 5 Explicar en el PR que se resolvió y por qué.

9. Reglas para AGENTS.md en Git

El repositorio debe explicar al agente cómo usar Git. Estas reglas evitan accidentes frecuentes: revertir trabajo ajeno, commitar secretos, usar `git add -A` sin revisar, hacer push directo o mezclar cambios no relacionados.

```
## Git rules for agents
```

- Always run ``git status`` before editing and before final response.
- Never run ``git reset --hard``, ``git clean -fd``, or checkout files unless explicitly asked.
- Do not use ``git add -A``; stage only files relevant to the task.
- Do not commit generated logs, screenshots, build output, `.env` files, or secrets.
- Prefer one branch per task; use worktrees for parallel agent sessions.
- Before opening PR, run lint/test/build required by this repo.
- PR description must include: summary, tests, risks, rollback, agent/tool used.
- Do not push to main. Do not deploy without human confirmation.

Buen habito

Si el agente encuentra cambios que no hizo, debe mencionarlos y evitar tocarlos. Esa regla protege trabajo humano y otros agentes.

10. Caso guia: cancelar reserva en MERN

Un equipo usa React, Express, MongoDB y JWT. Se pide agregar cancelacion de reserva con agente. Git debe ordenar el trabajo para que la revision sea simple.

Paso	Git	Evidencia
Preparar	git fetch; git worktree add ../reserva-cancel -b agent/reserva-cancel origin/main	Directorio aislado.
Implementar	Commits pequenos: backend, frontend, tests.	Diff revisable por capa.
Verificar	npm test; npm run build; prueba manual.	Resultados en PR.
Abrir PR	Descripcion con riesgo auth/datos.	Revisor sabe que mirar.
Revisar	CODEOWNERS para backend/auth.	Aprobacion humana antes de merge.
Cerrar	Eliminar worktree y rama si el PR se mergea.	Workspace limpio.

11. Checklist production ready

- La rama tiene un objetivo claro y pequeno.
- El agente no trabajo sobre main.
- No hay secretos, .env ni archivos generados innecesarios.
- El diff fue revisado completo antes del commit/PR.
- Los commits tienen mensajes utiles y cambios coherentes.
- Se ejecutaron pruebas relevantes y build.
- El PR explica riesgos, rollback y uso de agente.
- Cambios sensibles tienen reviewer/CODEOWNER.
- La rama base esta actualizada o el conflicto fue resuelto con pruebas.
- El worktree o rama temporal se limpio al terminar.

12. Fuentes oficiales consultadas

Material preparado el 23 de mayo de 2026. Para comandos exactos y opciones nuevas, verificar la documentacion vigente del proveedor antes de aplicarlo en repositorios reales.

Tema	Fuente
Git worktree 2.54	https://git-scm.com/docs/git-worktree
Git contributing workflows	https://git-scm.com/book/en/v2/Distributed-Git-Contributing-to-a-Project
GitHub protected branches	https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches

Tema	Fuente
GitHub CODEOWNERS	https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-code-owners
GitHub Actions secure use	https://docs.github.com/en/actions/reference/security/secure-use
Claude Code worktrees	https://code.claude.com/docs/en/worktrees
Codex AGENTS.md	https://developers.openai.com/codex/guides/agents-md
OpenCode rules	https://opencode.ai/docs/rules/

13. Cierre

Git es la capa de control que convierte trabajo agentico en ingenieria revisable. La velocidad del agente solo es valiosa si el historial queda limpio, las ramas son comprensibles, los PRs tienen evidencia, CI bloquea regresiones y las personas conservan la decision final sobre cambios sensibles.